Video Script: Advanced Seccomp - Troubleshooting and Best Practices

Module 3, Video 3 of 3
Estimated Duration: 14-15 minutes
Technical Setup: 1920x1080 @ 30fps, 150% zoom on all screens

Pre-Production Checklist

- [ ] PowerPoint slides (5 slides) ready
- [ ] auditd installed and running on host
- [ ] Terminal multiplexer (tmux) configured
- [ ] Kubernetes cluster (minikube/kind) with test deployment
- [ ] Sample broken Seccomp profiles prepared
- [ ] Production-ready profile examples downloaded
- [ ] Browser tabs: Kubernetes docs, production Seccomp examples from GitHub

---

VIDEO SCRIPT

SEGMENT 1: Introduction - The Reality of Seccomp in Production (0:00 - 1:30)

[SCREEN: PPT Slide 1 - Title slide with debugging theme]

NARRATION:
“Welcome to the final video in our Seccomp module. We’ve learned what Seccomp is and how to build custom profiles. But here’s what happens in the real world: you deploy your carefully crafted profile, everything seems fine, and then at 3 AM on a Saturday, your application breaks because it tried to make a syscall you didn’t account for.

[TRANSITION: PPT Slide 2 - Production incident timeline graphic]

Debugging Seccomp issues is challenging because:
- Error messages are often vague
- Failures only happen under specific conditions
- The kernel doesn’t always tell you which syscall was blocked

In this video, you’ll learn:
- How to read kernel audit logs to identify blocked syscalls
- Debugging techniques for broken profiles
- Strategies for balancing security and functionality
- Best practices for production deployments
- How to integrate Seccomp with Kubernetes

Let’s start with the most important skill: troubleshooting when things go wrong.”

SEGMENT 2: Anatomy of a Seccomp Failure (1:30 - 4:30)

[SCREEN: Terminal - fullscreen]

NARRATION:
“Let me show you a realistic scenario. I have a Node.js application that works fine with Docker’s default Seccomp profile, but I want to lock it down with a custom profile.”

[TYPE:]

```
cat app.js
```

[FILE DISPLAYS - simple Express.js app:]

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from secured app!');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

NARRATION:
“Simple Express server. I’ve created what I think is a complete Seccomp profile for it. Let’s try running it.”

[TYPE:]

```
docker run --rm --security-opt seccomp=seccomp-nodejs-broken.json \
  -p 3000:3000 \
  node:18-alpine \
  node /app/app.js
```

[COMMAND FAILS or APP HANGS]

[ERROR OUTPUT - something like: “Error: Cannot listen on port” or silent failure]

NARRATION:
“It failed, but the error doesn’t tell us which syscall was blocked. This is the frustrating part. Let me show you how to debug this properly.”

[SCREEN: Terminal split - 2 panes]

NARRATION:
“First, we need to enable audit logging. On the host system, I’ll check if auditd is running and configure it to log Seccomp denials.”

[LEFT PANE - TYPE:]

```
sudo systemctl status auditd
```

[OUTPUT shows auditd is active]

NARRATION:
“Good, auditd is running. Now I’ll configure it to log Seccomp events. Seccomp denials trigger audit log entries with type 1326.”

[TYPE:]

```
sudo auditctl -a exit,always -F arch=b64 -S all -F key=seccomp
```

NARRATION:
“This tells auditd to log all syscalls on 64-bit architecture with the ‘seccomp’ key. Now let’s run our broken container again.”

[RIGHT PANE - TYPE:]

```
docker run --rm --security-opt seccomp=seccomp-nodejs-broken.json \
  -p 3000:3000 \
  node:18-alpine \
  node /app/app.js
```

[APP FAILS AGAIN]

[LEFT PANE - TYPE immediately:]

```
sudo ausearch -m SECCOMP -ts recent | tail -20
```

[OUTPUT shows audit log entries - highlight key parts:]

```
type=SECCOMP msg=audit(1699123456.789:123): auid=1000 uid=0 gid=0 \
  ses=1 pid=12345 comm="node" exe="/usr/local/bin/node" \
  sig=31 arch=c000003e syscall=41 compat=0 ip=0x7f8a9b... code=0x7ffc0000
```

NARRATION:
“There it is! Look at the line that says `syscall=41`. That’s a syscall number, not a name. We need to translate it.”

[TYPE:]

```
ausyscall 41
```

[OUTPUT:]

```
socket
```

NARRATION:
“Ah! The `socket` syscall was blocked. Node.js needs this to create network sockets for the HTTP server. This is exactly what we were missing from our profile.”

SEGMENT 3: Fixing the Profile (4:30 - 7:00)

[SCREEN: Code Editor - seccomp-nodejs-broken.json]

NARRATION:
“Let’s add the missing syscall. Here’s our current profile—it has common syscalls but is missing networking-related ones.”

[SCROLL to syscalls array]

[HIGHLIGHT where socket should be added:]

NARRATION:
“I’ll add socket and related networking syscalls that Node.js needs.”

[ADD to the names array:]

```
"socket",
"setsockopt",
"bind",
"listen",
"accept",
"accept4",
```

[SAVE file as seccomp-nodejs-fixed.json]

[SCREEN: Terminal]

[TYPE:]

```
docker run --rm --security-opt seccomp=seccomp-nodejs-fixed.json \
  -p 3000:3000 -d \
  --name nodejs-secure \
  node:18-alpine \
  node /app/app.js
```

[CONTAINER STARTS SUCCESSFULLY]

NARRATION:
“It started! Let’s verify it’s working.”

[TYPE:]

```
curl http://localhost:3000
```

[OUTPUT:]

```
Hello from secured app!
```

NARRATION:
“Perfect! This is the iterative process: run, fail, check audit logs, add missing syscalls, repeat. Now let’s verify no more denials are happening.”

[TYPE:]

```
sudo ausearch -m SECCOMP -ts recent | grep nodejs
```

[OUTPUT shows no new denials]

NARRATION:
“Clean. No denials. The profile is working correctly.”

SEGMENT 4: Advanced Debugging Techniques (7:00 - 9:30)

[SCREEN: PPT Slide 3 - Debugging toolbox diagram]

NARRATION:
“Beyond audit logs, here are additional debugging techniques:

[BULLET POINTS APPEAR AS MENTIONED:]

1. **Verbose Logging:** Run your app with debug flags to see what it’s trying to do
2. **Strace Comparison:** Compare strace output with your profile to find gaps
3. **Permissive Mode:** Temporarily use SCMP_ACT_LOG instead of SCMP_ACT_ERRNO
4. **Kernel Messages:** Check dmesg for Seccomp-related kernel messages

Let me demonstrate permissive mode—this is incredibly useful.”

[SCREEN: Code Editor - create seccomp-nodejs-log.json]

NARRATION:
“I’ll create a version of our profile that logs denials instead of blocking them. This lets the app run while you discover what syscalls are being denied.”

[TYPE - showing the key difference:]

```
{
  "defaultAction": "SCMP_ACT_LOG",
  "architectures": [
    "SCMP_ARCH_X86_64"
  ],
  "syscalls": [
    {
      "names": ["exit", "exit_group", "read", "write"],
      "action": "SCMP_ACT_ALLOW"
    }
  ]
}
```

NARRATION (highlighting defaultAction):
“See? `SCMP_ACT_LOG` instead of `SCMP_ACT_ERRNO`. This means: allow everything, but log which syscalls would have been blocked.”

[SCREEN: Terminal]

[TYPE:]

```
docker run --rm --security-opt seccomp=seccomp-nodejs-log.json \
  node:18-alpine node /app/app.js
```

[APP RUNS SUCCESSFULLY]

NARRATION:
“The app works, and in the background, auditd is logging which syscalls aren’t in our profile. After running for a while, we can check the logs.”

[TYPE:]

```
sudo ausearch -m SECCOMP -ts recent | grep 'SCMP_ACT_LOG' | \
  awk '{print $NF}' | sort -u
```

[OUTPUT shows list of syscall numbers that were logged]

NARRATION:
“These are all the syscalls our app uses that we haven’t explicitly allowed. We can convert these to names and add them to a production profile. This is how you build comprehensive profiles without breaking your app during development.”

SEGMENT 5: Production Best Practices (9:30 - 12:00)

[SCREEN: PPT Slide 4 - Best practices checklist]

NARRATION:
“Now let’s talk about production-ready Seccomp strategies. Here are the key principles:

[POINTS APPEAR AS DISCUSSED:]

1. Start Permissive, Then Restrict

[SCREEN: Terminal]

NARRATION:
“Deploy with SCMP_ACT_LOG first in staging. Collect data for at least a week, covering all app functionality—scheduled jobs, error handling, admin tasks. Then build your production profile from that data.”

[SCREEN: PPT - continues]

2. Layer Your Policies

NARRATION:
“Don’t rely on Seccomp alone. Combine it with:
- AppArmor or SELinux (which we’ll cover in the next module)
- Read-only root filesystems
- User namespace isolation
- Network policies

Think defense in depth.”

3. Document Your Decisions

[SCREEN: Code Editor - showing commented profile]

NARRATION:
“Look at this production profile example:”

[DISPLAY:]

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "comment": "Custom profile for nginx v1.24 - Updated 2026-07-01",
  "architectures": ["SCMP_ARCH_X86_64", "SCMP_ARCH_X86", "SCMP_ARCH_X32"],
  "syscalls": [
    {
      "names": ["socket", "bind", "listen", "accept4"],
      "action": "SCMP_ACT_ALLOW",
      "comment": "Required for HTTP server functionality"
    },
    {
      "names": ["clone", "fork", "vfork"],
      "action": "SCMP_ACT_ALLOW",
      "comment": "Worker process spawning - needed for multi-worker mode"
    }
  ]
}
```

NARRATION:
“See the comments? They explain WHY each syscall is allowed. Six months from now when you’re updating nginx, you’ll thank yourself for this documentation.”

[SCREEN: PPT Slide 5 - Maintenance strategy]

4. Version and Test

NARRATION:
“Treat Seccomp profiles like code:
- Store in version control
- Pin to application versions
- Test profile updates in CI/CD
- Have rollback procedures

When your application updates, re-run strace and validate the profile still covers all functionality.”

5. Monitor and Alert

[SCREEN: Browser - showing sample Prometheus/Grafana dashboard]

NARRATION:
“In production, monitor Seccomp denials. Set up alerts for unexpected syscall blocks. This helps you catch issues before they become incidents.”

SEGMENT 6: Seccomp in Kubernetes (12:00 - 14:00)

[SCREEN: Code Editor - Kubernetes YAML]

NARRATION:
“Finally, let’s see how to use Seccomp profiles in Kubernetes. Kubernetes has built-in support for Seccomp, and it’s easier than you might think.”

[DISPLAY Kubernetes Pod manifest:]

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-secure
spec:
  securityContext:
    seccompProfile:
      type: Localhost
      localhostProfile: profiles/nginx-custom.json
  containers:
  - name: nginx
    image: nginx:alpine
    ports:
    - containerPort: 80
```

NARRATION (explaining key parts):
“The `securityContext` at the pod level sets the Seccomp profile.
- `type: Localhost` means use a custom profile from the node
- `localhostProfile` specifies the path relative to the kubelet’s Seccomp directory

You need to place your JSON profile on each node at `/var/lib/kubelet/seccomp/profiles/nginx-custom.json`.”

[SCREEN: Terminal]

NARRATION:
“Kubernetes also supports the RuntimeDefault profile, which uses the container runtime’s default—usually Docker or containerd’s default.”

[SHOW alternative YAML:]

```yaml
securityContext:
  seccompProfile:
    type: RuntimeDefault
```

NARRATION:
“This is a good starting point. It’s better than nothing, but custom profiles are more secure.”

[TYPE command to apply:]

```
kubectl apply -f nginx-secure-pod.yaml
```

[POD STARTS]

[TYPE:]

```
kubectl get pod nginx-secure -o jsonpath='{.spec.securityContext.seccompProfile}'
```

[OUTPUT shows profile configuration]

NARRATION:
"There's our profile configuration. One challenge with Kubernetes is profile management at scale. Solutions include:
- ConfigMaps to distribute profiles
- Init containers to fetch profiles from a central repo
- Admission controllers to enforce Seccomp requirements
- Tools like Security Profiles Operator for automated management

The Security Profiles Operator is particularly powerful—it can record profiles from running workloads, similar to our strace approach but automated."

SEGMENT 7: Lab Exercise and Real-World Scenarios (14:00 - 14:45)

[SCREEN: PPT Slide 6 - Lab overview]

NARRATION:
"Now for your final lab in this module: Lab 3.3 and 3.4.

Lab 3.3 - You're given a multi-container application that's failing with a Seccomp profile. Using the troubleshooting techniques we covered:
- Enable audit logging
- Identify blocked syscalls
- Fix the profile
- Validate all functionality works

Lab 3.4 - Production readiness exercise:
- Deploy an app with SCMP_ACT_LOG
- Run comprehensive tests
- Build a production profile
- Deploy to Kubernetes
- Set up monitoring

You'll also get a bonus challenge: a profile that's TOO permissive. Your job is to identify unnecessary syscalls and remove them without breaking functionality."

SEGMENT 8: Module Wrap-Up and Next Steps (14:45 - 15:30)

[SCREEN: PPT Slide 7 - Module summary and next steps]

NARRATION:
"Let's recap this entire Seccomp module:

Video 1: We learned what Seccomp is, how it filters syscalls, and analyzed Docker's default profile.

Video 2: We built custom profiles using strace, reducing attack surface by 80%+.

Video 3: We mastered troubleshooting with audit logs, explored best practices, and deployed to Kubernetes.

Key Takeaways:
✅ Seccomp is your most powerful tool for syscall filtering
✅ Custom profiles dramatically reduce attack surface
✅ Audit logs are essential for debugging
✅ Start with permissive mode, then restrict
✅ Document, version, and test your profiles
✅ Seccomp works seamlessly with Kubernetes

What's Next?

In Module 4, we'll explore AppArmor and SELinux—Mandatory Access Control systems that work alongside Seccomp. While Seccomp controls syscalls, MAC systems control file access, process interactions, and more.

Together, Seccomp + MAC creates a nearly impenetrable security layer.

See you in the next module!"

[FADE OUT]

POST-PRODUCTION NOTES

B-Roll Suggestions:

- Animated audit log flow diagram
- Screen recording of real audit logs scrolling
- Kubernetes dashboard showing pod security
- Side-by-side: broken vs. fixed profile comparison

Audio Notes:

- Use suspenseful music during the "failure" section
- Add triumphant tone when profile is fixed
- Boost volume on audit log explanations
- Add subtle "typing" sound effects during code editing

Text Overlays:

- Large syscall names when first identified in logs
- "BEFORE" and "AFTER" labels during fix demonstration
- Step-by-step troubleshooting checklist that builds as you go
- Key command syntax for ausearch and ausyscall

Timing Checkpoints:

- 0:00-1:30: Intro (1.5 min)
- 1:30-4:30: Failure scenario (3 min)
- 4:30-7:00: Fixing the profile (2.5 min)
- 7:00-9:30: Advanced debugging (2.5 min)
- 9:30-12:00: Best practices (2.5 min)

- 12:00-14:00: Kubernetes integration (2 min)
- 14:00-14:45: Lab intro (0.75 min)
- 14:45-15:30: Wrap-up (0.75 min)
- **Total: 15:30** (within 14-15 min target)

Files Needed:

- `slides_module3_video3.pptx` (7 slides)
- `seccomp-nodejs-broken.json`
- `seccomp-nodejs-fixed.json`
- `seccomp-nodejs-log.json`
- `nginx-secure-pod.yaml`
- `app.js` (sample Node.js app)
- `lab_3.3_instructions.md`
- `lab_3.4_instructions.md`
- `lab_validation.sh`
- Sample production profiles from real projects (github.com examples)

INSTRUCTOR NOTES

Common Issues:
- auditd may not be installed - install with `apt-get install auditd`
- ausearch requires root/sudo access
- Kubernetes Seccomp profiles must exist on ALL nodes in the cluster
- syscall numbers differ between architectures (x86 vs ARM)

Variation Options:
- Can demonstrate with Podman instead of Docker
- Can show OCI runtime integration
- Advanced: demonstrate Security Profiles Operator in Kubernetes
- Can show integration with Falco for runtime monitoring

Accessibility:
- Slow down during audit log analysis—this is complex
- Highlight specific syscall numbers in terminal output
- Use color coding for “blocked” vs “allowed” syscalls
- Repeat key commands before executing

Pro Tips for Delivery:
- Pre-configure auditd to avoid live setup issues
- Have working profiles as backup files
- Test Kubernetes demo in advance
- Keep a syscall reference chart visible for Q&A
- Mention that production companies like Google, Netflix have published profiles as examples

Discussion Points (if time permits):
- Trade-offs: security vs. maintainability
- Seccomp vs. eBPF for syscall filtering
- How Seccomp interacts with ptrace and debugging

- Seccomp in rootless containers
- Performance impact (minimal, usually <1%)

Real-World Examples to Mention:
- Chrome browser uses Seccomp for sandboxing
- systemd uses Seccomp for service isolation
- Docker/containerd default profiles used by millions of containers
- Kubernetes PodSecurityStandards include Seccomp requirements